

Gen AI Academy

APAC Edition

Build in APAC. Build for the world.



PARTICIPANT DETAILS

- a. Participant name : **Hariharan C**
- b. Problem Statement : **Build AI-powered applications using AI-ready databases like AlloyDB.**
- c. Github repo link : <https://github.com/BlackWizard123/Track-3---Book-store-NL2SQL-GCP>
- d. Cloud Run URL : <https://bookstore-api-1059652519537.us-central1.run.app/>

Brief about the idea

- **Bookstore AI** - Bookstore database hosted on **AlloyDB (PostgreSQL)**
- Users query the database using plain English — no SQL needed
- **Gemini AI** converts the question into a valid SQL query
- SQL executes live on AlloyDB and returns real results
- Gemini generates a human-friendly answer from the results
- Delivered via a clean chat interface deployed on **Cloud Run**

Meeting the Build Criteria

How did you navigate the requirements of your chosen track to build a custom solution - using ADK, MCP, or AlloyDB AI that effectively addresses the 'What You Must Build' criteria?

- Custom bookstore dataset created from scratch (authors, books, orders)
- Table schemas designed and populated manually in AlloyDB
- Gemini AI integrated to enable natural language querying
- Every user question is converted to SQL and executed live
- Results passed back to Gemini to generate meaningful responses
- Fully deployed end-to-end on Google Cloud — not just a local demo

Opportunities

How different is it from any of the other existing ideas?

Most NL-to-SQL tools are generic or demo-only. This is a fully deployed, end-to-end working application on Google Cloud with a real database, real queries, and real answers — not a prototype.

How will it be able to solve the problem?

Removes the SQL barrier entirely. Any non-technical user — a store manager, analyst, or student — can get data insights instantly just by asking a question in plain English.

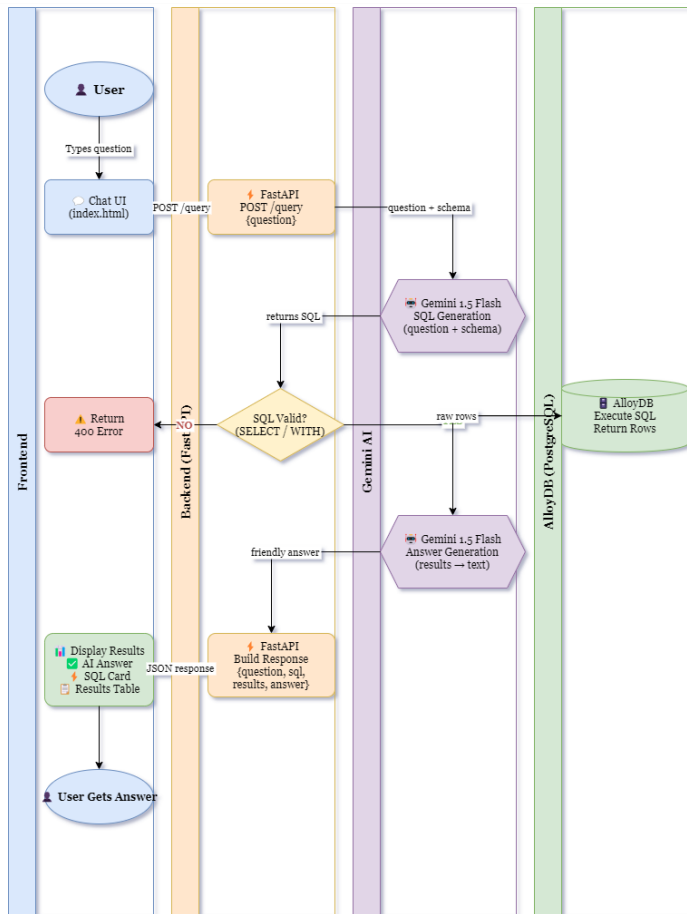
USP of the proposed solution

The combination of AlloyDB + Gemini gives both enterprise-grade database reliability and state-of-the-art AI understanding in a single lightweight API — deployable in minutes on Cloud Run.

List of features offered by the solution

- Natural language to SQL conversion
- Live query execution on **AlloyDB**
- AI-generated human-friendly answers
- Generated SQL visible for transparency
- Results displayed as a formatted table
- Health check with live DB status
- Suggestion chips for quick queries
- Deployable via **Google Cloud Run**

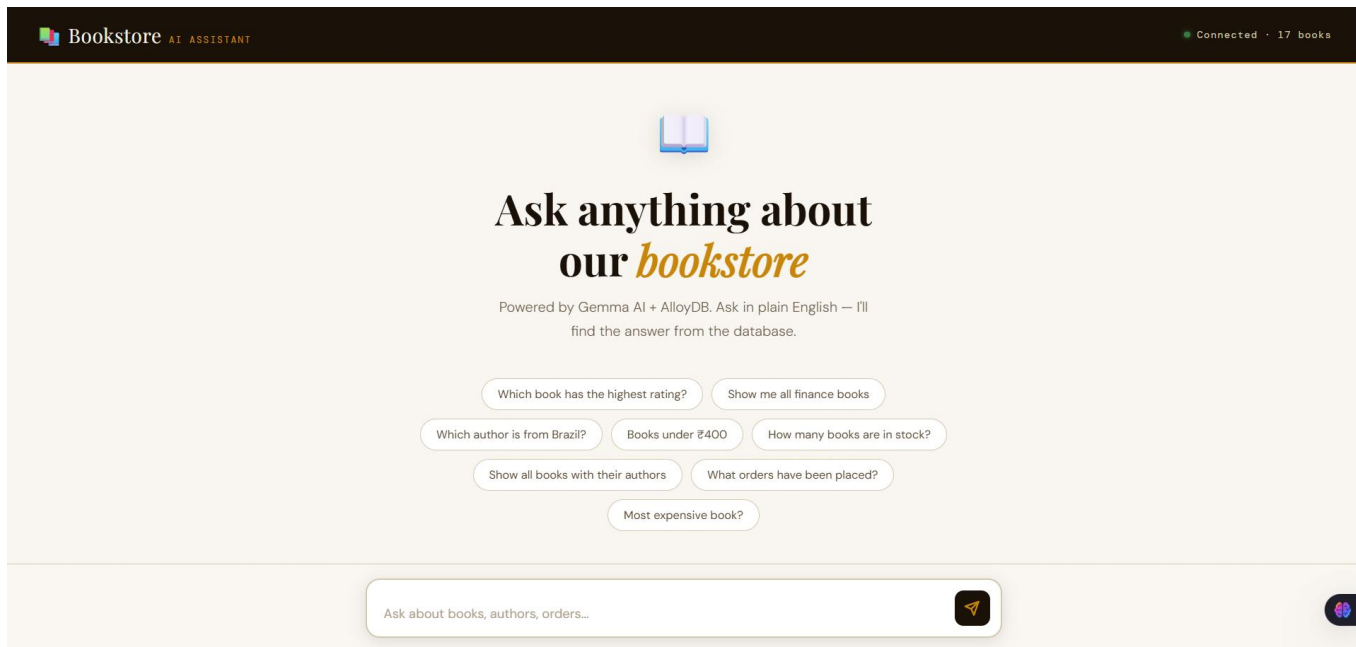
Process flow diagram or Use-case diagram



Technologies used in the solution

- **AlloyDB** — PostgreSQL database on Google Cloud
- **Gemini 1.5 Flash** — NL-to-SQL + answer generation
- **FastAPI** — backend API and query orchestration
- **psycopg2** — AlloyDB connection and query execution
- **Cloud Run** — serverless deployment
- **HTML/CSS/JS** — chat interface

Snapshots of the prototype



<https://bookstore-api-1059652519537.us-central1.run.app/>

The screenshot displays the 'Bookstore AI Assistant' interface. At the top, it says 'Bookstore AI ASSISTANT' and 'Connected · 17 books'. The user's prompt is 'Show me all finance books'. The assistant's response is: 'Okay, here are the finance books we have in stock! You'll find "Rich Dad Poor Dad" and "The Psychology of Money" available, with varying quantities and prices. Let me know if you'd like to know more about any of them!'. Below this, the 'GENERATED SQL' is shown: `SELECT * FROM books WHERE genre = 'Finance'`. The results are displayed in a table with 4 rows and 7 columns: ID, TITLE, AUTHOR_ID, GENRE, PRICE, RATING, and STOCK. The data rows are: (2, 'Rich Dad Poor Dad', 2, 'Finance', 399.00, 4.7, 15), (4, 'The Psychology of Money', 4, 'Finance', 499.00, 4.8, 12), (7, 'Rich Dad Poor Dad', 2, 'Finance', 399.00, 4.7, 30), and (10, 'The Psychology of Money', 5, 'Finance', 499.00, 4.8, 40). The user's next prompt is 'What is the most expensive book in the shop?'. The assistant's response is: 'Looks like "Harry Potter and the Philosopher's Stone" is currently the most expensive book we have in stock! It's a classic, of course. Happy reading!'. The 'GENERATED SQL' is: `SELECT title FROM books ORDER BY price DESC LIMIT 1`. The result is shown in a table with 1 row and 1 column: TITLE, with the value 'Harry Potter and the Philosopher's Stone'. At the bottom, there is a text input field with the placeholder 'Ask about books, authors, orders...' and a microphone icon.

Bookstore AI ASSISTANT Connected · 17 books

Show me all finance books

Okay, here are the finance books we have in stock! You'll find "Rich Dad Poor Dad" and "The Psychology of Money" available, with varying quantities and prices. Let me know if you'd like to know more about any of them!

GENERATED SQL

```
SELECT * FROM books WHERE genre = 'Finance'
```

4 RESULTS

ID	TITLE	AUTHOR_ID	GENRE	PRICE	RATING	STOCK
2	Rich Dad Poor Dad	2	Finance	399.00	4.7	15
4	The Psychology of Money	4	Finance	499.00	4.8	12
7	Rich Dad Poor Dad	2	Finance	399.00	4.7	30
10	The Psychology of Money	5	Finance	499.00	4.8	40

What is the most expensive book in the shop?

Looks like "Harry Potter and the Philosopher's Stone" is currently the most expensive book we have in stock! It's a classic, of course. Happy reading!

GENERATED SQL

```
SELECT title FROM books ORDER BY price DESC LIMIT 1
```

1 RESULT

TITLE
Harry Potter and the Philosopher's Stone

Ask about books, authors, orders...

<https://bookstore-api-1059652519537.us-central1.run.app/>

Google Cloud 

Gen AI Academy

APAC Edition

Thank you

Build in APAC. Build for the world.

